SkillSync

AI-Assisted Career Navigation for Undergraduate Tech Curricula

Laboratory / project documentation

1) Title of the project

SkillSync is a web application that combines rule-based career matching with LLM-generated learning roadmaps so undergraduates can align coursework, skills, and interests with industry-oriented technology careers.

2) Introduction

University programs often cover theory and breadth, while employers expect demonstrable skills, projects, and role-specific tooling. Students frequently lack a structured bridge between transcript, self-reported skills, and realistic job targets. SkillSync addresses this by ingesting a student profile (major, semester, skills, interests, selected courses), scoring predefined career paths with a transparent rule engine, persisting results, and optionally invoking a large language model (Groq) to produce phased roadmaps, visual guides, and a context-aware chat assistant. A parallel intake supports learners from non-technical backgrounds targeting tech-adjacent roles.

3) Problem statement and idea description

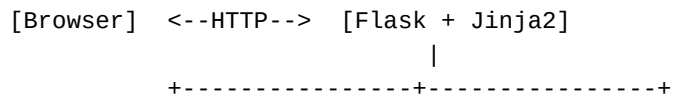
Problem: (i) Weak visibility into how courses map to employable skills; (ii) generic career advice without curriculum context; (iii) fragmented planning across free resources and degree requirements.

Idea: Encode career requirements and a course-to-skill catalog in structured data; score careers deterministically for explainability; use the LLM only for narrative roadmaps and tutoring where creativity and personalization add the most value. Curriculum coverage reduces duplicate recommendations and steers the model toward projects for skills already scheduled in upcoming terms.

4) Project architecture

Client: HTML templates, CSS (landing, dashboard, resources), and JavaScript for interactive UI (e.g., roadmap guide, resources). Server: Flask application factory with blueprints for authentication, profile, career analysis, progress, chat, non-tech flows, resources, dashboard, CV helpers, chatbot API, and news. Persistence: SQLite via Flask-SQLAlchemy (users, profiles, courses, skills, career paths, scores, roadmaps, milestones, chat messages, feedback). Engines: curriculum_engine (JSON catalog crosswalk), rule_engine (scoring), decision_engine (comparisons / fit), llm_engine (Groq prompts for roadmaps and guides). Configuration: environment variables for SECRET_KEY, GROQ_API_KEY, optional GNews.

Layered view (conceptual):



```
        v                 v                 v
[SQLite ORM]    [Rule + Curriculum]   [Groq LLM]
        |                 |                 |
skillsync.db    career_paths.json   roadmaps / chat
                course_catalog.json
```

Target population:

- Undergraduate students in computing-related majors building a tech career plan.
- Students who can enumerate completed/in-progress/upcoming courses from a catalog.
- Non-technical background users re-skilling toward digital, creative, or IT-adjacent roles.
- Educators or counselors (role field exists) as secondary stakeholders for future use.

Platform: Python 3.x, Flask 3.x, Flask-Login, Werkzeug; local deployment on http://127.0.0.1:5000 (development). Data: JSON assets under /data; uploads for CV. External APIs: Groq (primary LLM), optional OpenRouter in code paths, optional GNews for news.

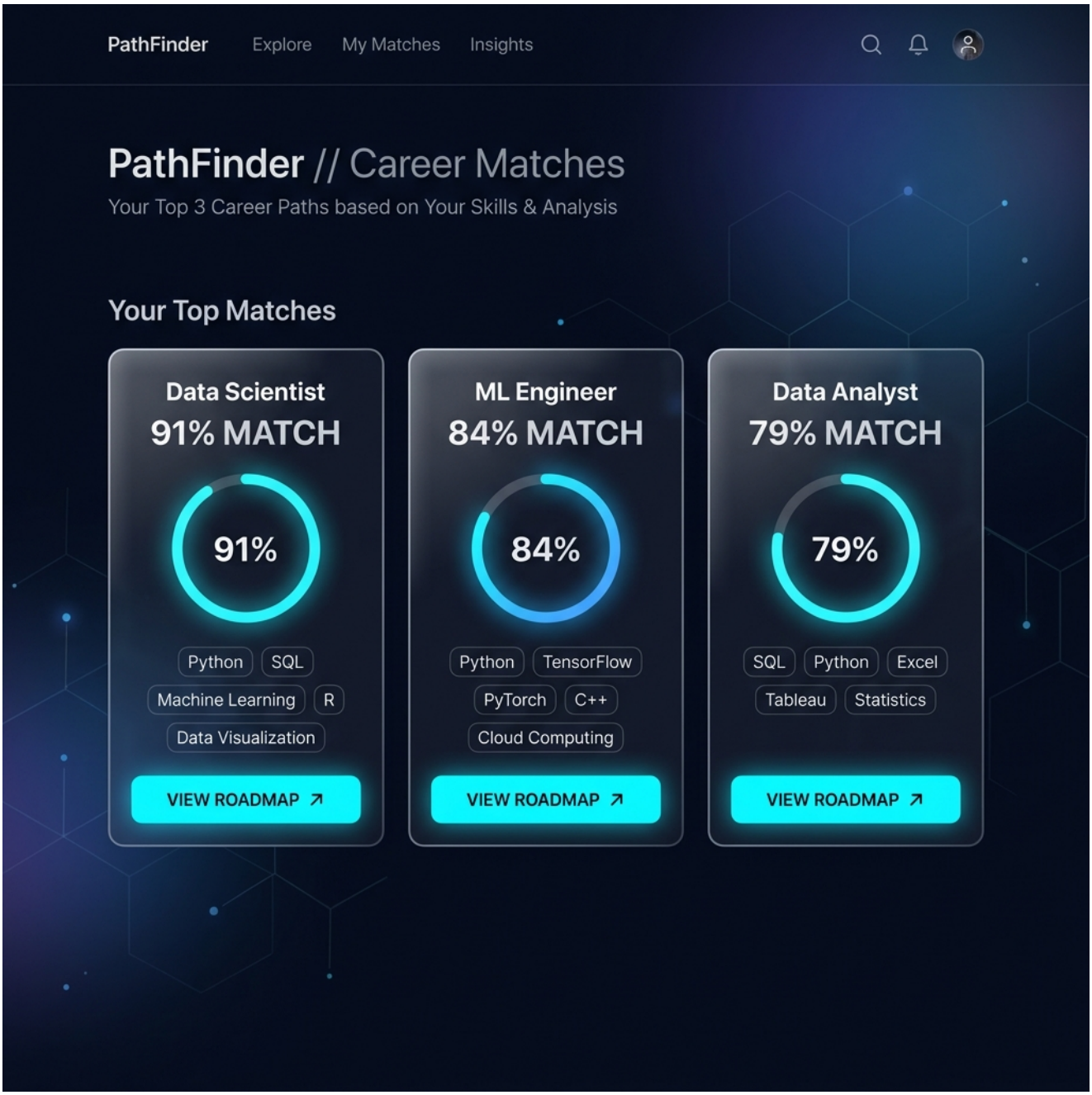
5) Project features

- Registration, login, and session management with password hashing.
- Tech profile setup: major, semester, location, weekly hours, interests, skills, course selection.
- Career analysis: run rule engine, persist CareerScore rows, bucket results (best / adjacent / stretch / wildcard).
- Career overview, compare paths, and decision-engine-backed comparisons.
- LLM-generated phased roadmaps with milestones (core, project, resource, checkpoint) and milestone tracking.
- Visual knowledge-style guide generation for roadmaps.
- Per-roadmap chat history tied to milestones and Groq.
- Non-tech intake and AI roadmap for alternative learner personas.
- Dashboard metrics, resources area, tech news, CV-related routes, sidebar career chatbot.

6) Project screenshots (marketing / UI mockups from repository)

Dashboard concept

Career matching concept


PathFinder
Explore
My Matches
Insights
PathFinder // Career Matches
Your Top 3 Career Paths based on Your Skills & Analysis
Your Top Matches
Data Scientist
91% MATCH
91%
Python
SQL
Machine Learning
R
Data Visualization
VIEW ROADMAP
ML Engineer
84% MATCH
84%
Python
TensorFlow
PyTorch
C++
Cloud Computing
VIEW ROADMAP
Data Analyst
79% MATCH
79%
SQL
Python
Excel
Tableau
Statistics
VIEW ROADMAP

7) Source code excerpt (rule engine: ranking logic)

The rule engine (engines/rule_engine.py) maps interests and activities to career slugs, merges explicit skill proficiency with curriculum-derived coverage from course_catalog.json, and produces a 0-100 score with explainable subcomponents. The excerpt below shows the public ranking entry point.

```
def rank_careers(profile, career_paths, user_courses, min_score=5):
    results = []
    for career in career_paths:
        details = score_career_path(profile, career, user_courses)
        if details["score"] >= min_score:
            results.append((career, details))
    results.sort(key=lambda x: x[1]["score"], reverse=True)
    return results

# score_career_path combines (up to 100):
# - skill_match_score: explicit skills + curriculum (40)
# - interest_score: INTEREST_CAREER_MAP (35)
# - activity_score: ACTIVITY_CAREER_MAP (15)
# - curriculum_bonus + semester_boost (10 + 0-8, capped at 100)
```

Full implementation: engines/rule_engine.py, engines/curriculum_engine.py.

8) Social and economic value

Social: Improves equity of career information for students without professional networks; supports non-traditional paths into tech-adjacent work; encourages structured self-study and mental health-friendly planning via phased goals.

Economic: Better alignment of human capital with labor-market demand can shorten job-search friction, reduce skills mismatches, and increase productivity of graduates entering software, data, cloud, security, and product roles. Institutions may use similar systems to scale advising.

9) Risk management

- LLM hallucination / stale advice: mitigate with structured JSON expectations, curriculum constraints in prompts, and human review for high-stakes decisions.
- API key exposure: store secrets only in .env, never commit; rotate keys; use HTTPS in production.
- Privacy: student profiles and chat logs are sensitive; encrypt at rest in production, minimize retention, and apply access control (Flask-Login already scopes data by user).
- Scoring bias: interest maps are hand-curated; periodic audit against labor statistics and diverse role titles.
- Dependency on third-party Groq availability: graceful error handling and user-visible messages when keys are missing or rate-limited.
- SQLite concurrency limits: migrate to PostgreSQL if deployed multi-user at scale.

10) Conclusion

SkillSync demonstrates a pragmatic hybrid architecture: explainable rules for ranking and gap analysis, plus generative AI for rich roadmaps and conversational help. By grounding

recommendations in a course catalog and student selections, the system reduces redundant study plans and better reflects each learner's academic trajectory. Future work includes production hardening, broader catalogs, empirical validation with students, and optional integration with institutional LMS data.

11) References (IEEE style)

[1] Pallets Projects, "Flask Documentation," Flask, 2024. [Online]. Available: https://flask.palletsprojects.com/

[2] Groq Inc., "GroqCloud API Reference," Groq Developer Docs, 2024. [Online]. Available: https://console.groq.com/docs

[3] M. A. Garcia and S. N. Jones, "SQLite," in SQLite Documentation, D. R. Hipp, Ed., SQLite Consortium, 2024. [Online]. Available: https://www.sqlite.org/docs.html

[4] SQLAlchemy authors, "SQLAlchemy Documentation," SQLAlchemy, 2024. [Online]. Available: https://docs.sqlalchemy.org/

[5] OpenAI, "OpenAI Python API library used with compatible endpoints," OpenAI GitHub, 2024. [Online]. Available: https://github.com/openai/openai-python

[6] W3C, "HTML Living Standard," WHATWG/W3C, 2024. [Online]. Available: https://html.spec.whatwg.org/